

Docker-compose e Kubernetes(K8s)

Guia Prático de Orquestração de Contêineres



Para futuras consultas aos comandos, o PDF está disponível para download.

Todo o material e arquivos desenvolvidos durante esta aula podem ser encontrados no seguinte repositório do GitHub:

<https://github.com/kelvinalmeida/Aula-Docker>.

Instale o Docker Desktop pelo link:

<https://www.docker.com/>

1. Rodando os containers utilizando docker-compose.yml

Lembra do que aconteceu na aula passada?

- Criamos 3 imagens no docker

```
PS C:\Users\kelvi\Desktop\Aula docker> docker images
REPOSITORY          TAG          IMAGE ID      CREATED      SIZE
servico_catalogo    latest      41dae9d32492  20 hours ago 208MB
mine_netflix        latest      58c622482f63  20 hours ago 208MB
servico_assinatura  latest      51409552f23e  2 days ago   208MB
PS C:\Users\kelvi\Desktop\Aula docker> |
```

- Criamos uma rede chamada mine_netflix para que os containers se comuniquem.

```
PS C:\Users\kelvi\Desktop\Aula docker> docker network ls
NETWORK ID          NAME          DRIVER        SCOPE
72f4ae7e89f1        bridge        bridge         local
8a91141e97c6        host          host           local
386b0775c700        netflix-rede  bridge         local
24db715b79c1        none          null           local
PS C:\Users\kelvi\Desktop\Aula docker> |
```

- Em seguida criamos os containers com os comandos:

```
docker run -d -p 5001:5001 --network netflix-rede --name
cont_servico_assinatura servico_assinatura
```

```
docker run -d -p 5002:5002 --network netflix-rede --name  
cont_servico_catalogo servico_catalogo
```

```
docker run -d -p 5000:5000 --network netflix-rede --name  
cont_mine_netflix mine_netflix
```

Perceba a quantidade de passos necessários para que os *containers* estivessem prontos para execução. Imagine se o nosso sistema tivesse, por exemplo, 10 *containers*? Seria extremamente trabalhoso repetir todos esses passos para cada um deles.

É exatamente para resolver essa complexidade que existe o *docker-compose*, o qual será o tema da nossa aula de hoje.

O Conceito (A Analogia)

"A Diferença entre o Músico Solista e a Orquestra"

Até agora, nós se comportamos como um produtor que precisa ligar para cada músico individualmente:

1. "Alô, Guitarrista (Catálogo)? Venha para o estúdio." (docker run catálogo)
2. "Alô, Baterista (Assinatura)? Venha para o estúdio." (docker run assinatura)
3. "Alô, Vocalista (Gateway)? Pode vir." (docker run gateway)
4. "Ah, e não esqueçam de se plugarem na mesma tomada!" (docker network)

O Problema: Se o show tiver 50 músicos, você vai ficar louco ligando um por um. E se um deles cair? E se a ordem importar?

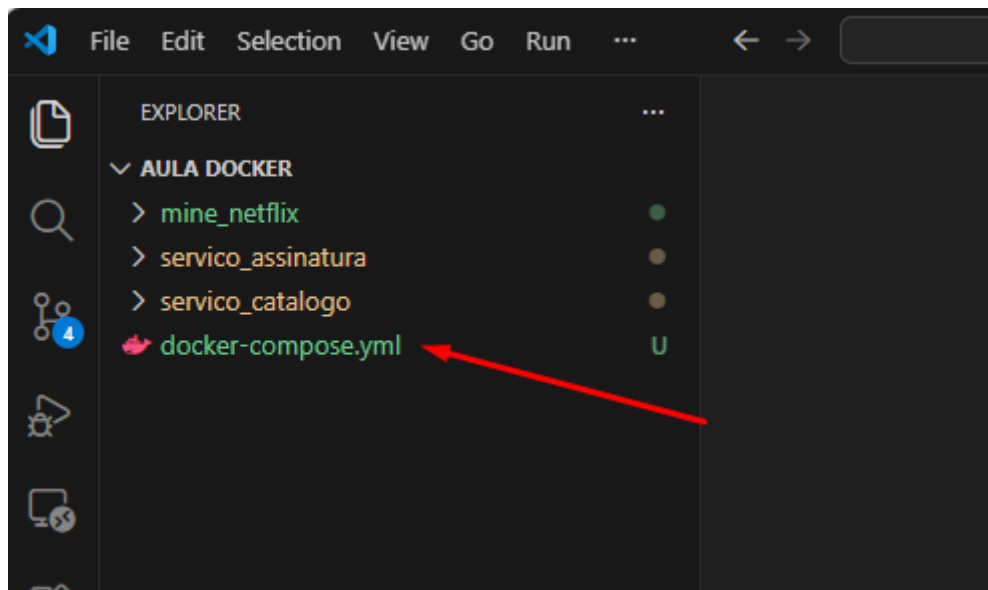
A Solução: O Docker Compose (O Maestro)

O Docker Compose é um arquivo de texto (uma "partitura") onde você escreve **uma única vez** tudo o que precisa acontecer.

- Você entrega a partitura para o Docker.
- O Docker lê: "Ah, preciso de um catálogo na porta 5002, uma assinatura na 5001 e um gateway na 5000, todos na mesma rede".
- Com **um único comando**, a banda toda começa a tocar junta.

A Prática (O Arquivo Mágico)

Para começarmos, iremos criar um arquivo chamado **docker-compose.yml** na raiz do projeto, exatamente onde estão as pastas **mine_netflix**, **servico_assinatura**, etc.



Abaixo está o código do **docker-compose.yml** (Explicado linha a linha):

```
services:
  # 1. Serviço de Assinatura
  cont_servico_assinatura:
    build: ./servico_assinatura    # Onde está o Dockerfile? Na pasta
servico_assinatura
    ports:
      - "5001:5001"    # Porta PC : Porta Container
    container_name: container_assinatura

  # 2. Serviço de Catálogo
  cont_servico_catalogo:
    build: ./servico_catalogo    # Onde está o Dockerfile? Na pasta servico_catalogo
    ports:
      - "5002:5002"
    container_name: container_catalogo

  # 3. Gateway (O Maestro)
  mine_netflix:
    build: ./mine_netflix    # Onde está o Dockerfile? Na pasta mine_netflix
    ports:
      - "5000:5000"
    container_name: container_gateway
    # O Pulo do Gato: Dizemos que o Gateway "depende" dos outros para nascer
    depends_on:
      - cont_servico_assinatura
```

```
- cont_servico_catalogo
```

Perceba que no código acima temos o **cont_servicoassinatura** e **cont_servicocatalogo** e **mine_netflix**, eles são os mesmos serviços (containers) da aula anterior.

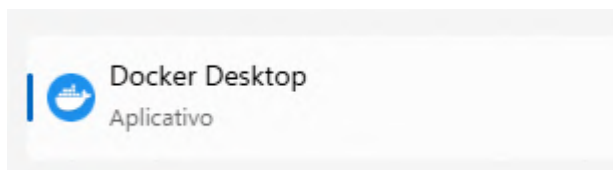
A principal vantagem aqui é que a criação da rede se torna desnecessária, pois o próprio Docker Compose se encarrega de configurá-la automaticamente.

Configuramos tudo que precisamos em um único arquivo `docker-compose.yml` e vamos executar todos os containers com um único comando.

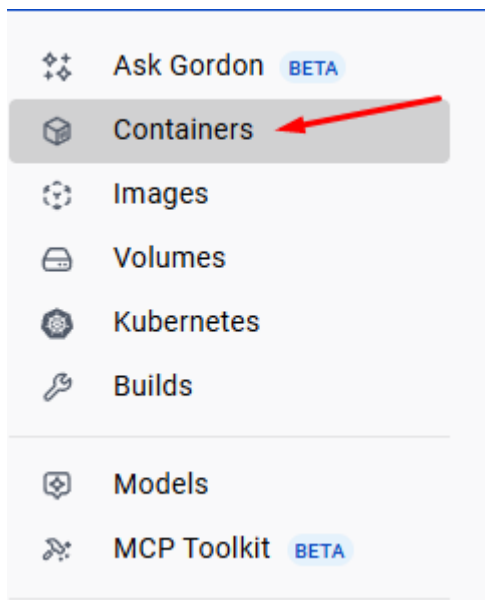
Deletando os containers anteriores

Para evitar conflitos indesejados, é crucial excluir os containers e imagens criados anteriormente antes de prosseguir com a execução do `docker-compose.yml`.

- Abra o Docker Desktop



- Clique em containers



- Delete todos os containers

Container CPU usage ⓘ
0.05% / 1200% (12 CPUs available)

Container memory usage ⓘ
75.29MB / 7.52GB

Show charts

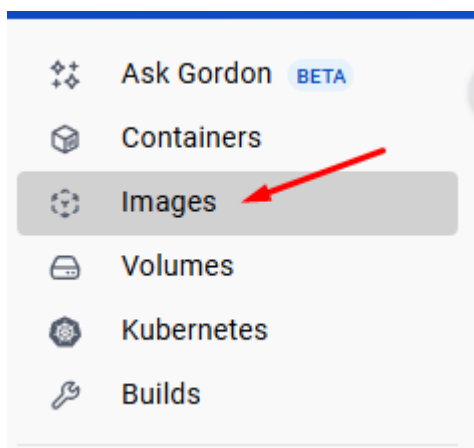
Q Search

Only show running containers

Delete

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Disk read/wr	Actions
☒	cont_mine_netfl	589aa0c2561b	mine_netfli	5000:5000	0.02%	27.52MB / 7.7GB	0.35%	7.81MB / 256	☐ ⋮
☒	cont_servico_as	0668023daac3	servico_as	5001:5001	0.01%	23.88MB / 7.7GB	0.3%	4.51MB / 256	☐ ⋮
☒	cont_servico_ca	988113e7307c	servico_cat	5002:5002	0.02%	23.89MB / 7.7GB	0.3%	4.51MB / 256	☐ ⋮

- Depois vá em imagens e delete todas as imagens



Local My Hub

194.94 MB / 15.83 GB in use 3 Images

Last refresh: 36 minutes ago ↻

Q Search

Delete Space to be reclaimed 194.94 MB

	Name	Tag	Image ID	Created	Size	Actions
☒	mine_netflix	latest	a69d1658eed3	2 days ago	208.39 MB	☐ ⋮
☒	servico_assinatura	latest	d2fc36f2d0d7	4 days ago	208.38 MB	☐ ⋮
☒	servico_catalogo	latest	10b72cc4b189	35 minutes ag	208.38 MB	☐ ⋮

Executando

2. O Comando Único (Up):

Abra o terminal e navegue para a pasta onde está o arquivo `docker-compose.yml`:

```
PS C:\Users\kelvi\Desktop\Aula docker> ls

Directory: C:\Users\kelvi\Desktop\Aula docker

Mode                LastWriteTime         Length Name
----                -
d-----          02/02/2026   13:49          mine_netflix
d-----          01/02/2026   14:59        servico_assinatura
d-----          01/02/2026   14:59        servico_catalogo
-a---          06/02/2026   09:31          868 docker-compose.yml
```

Execute o comando:

```
docker-compose up --build
```

- **up**: Subir tudo.
- **--build**: "Por favor, se eu mudei o código, reconstrua as imagens antes de subir."

Resultado Esperado:

O terminal vai virar uma "chuva de logs" colorida. Cada cor é um serviço diferente respondendo ao mesmo tempo.

```
PS C:\Users\kelvi\Desktop\Aula docker> docker-compose up --build
[+] Building 6.4s (25/25) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 1.67kB                                         0.0s
=> [cont_servico_assinatura internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 749B                                       0.0s
=> [cont_servico_catalogo internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 751B                                       0.0s
=> [gateway internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 748B                                       0.0s
=> [cont_servico_assinatura internal] load metadata for docker.io/library/python:3.9-slim 4.9s
=> [cont_servico_catalogo internal] load .dockerignore                  0.0s
=> => transferring context: 2B                                             0.0s
=> [cont_servico_assinatura internal] load .dockerignore                0.0s
=> => transferring context: 2B                                             0.0s
=> [gateway internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                             0.0s
=> [cont_servico_assinatura 1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b 0.0s
=> resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b 0.0s
```

```

Attaching to container_assinatura, container_catalogo, container_gateway
container_assinatura | * Serving Flask app 'assinatura_service.py'
container_assinatura | * Debug mode: off

container_catalogo | * Serving Flask app 'catalogo_service.py'
container_catalogo | * Debug mode: off
container_assinatura | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
container_assinatura | * Running on all addresses (0.0.0.0)

container_catalogo | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
container_catalogo | * Running on all addresses (0.0.0.0)

container_assinatura | * Running on http://172.19.0.3:5001
container_catalogo | * Running on http://127.0.0.1:5002

container_assinatura | Press CTRL+C to quit
container_catalogo | * Running on http://172.19.0.2:5002
container_catalogo | Press CTRL+C to quit
container_gateway | * Serving Flask app 'gateway_netflix.py'
container_gateway | * Debug mode: off
container_gateway | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
container_gateway | * Running on all addresses (0.0.0.0)
container_gateway | * Running on http://127.0.0.1:5000
container_gateway | * Running on http://172.19.0.4:5000
container_gateway | Press CTRL+C to quit

```

Na imagem acima os contêineres **container_catalogo** (amarelo), **container_assinatura** (azul) e **container_gateway** (verde) estão operacionais. Todos estão em execução em suas portas designadas e compartilham a mesma rede definida pelo arquivo `docker-compose.yml`. Inclusive é nessa tela que irão aparecer os logs e erros relacionados a o containers.

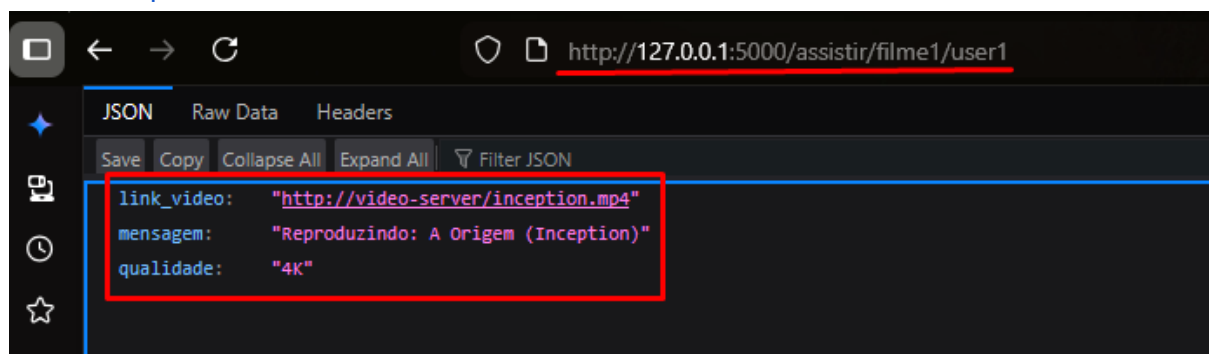
Não foi simples executar todos os *containers* com apenas um comando? Gerenciar um volume maior de *containers* se torna muito mais fácil agora.

🔧 Testando

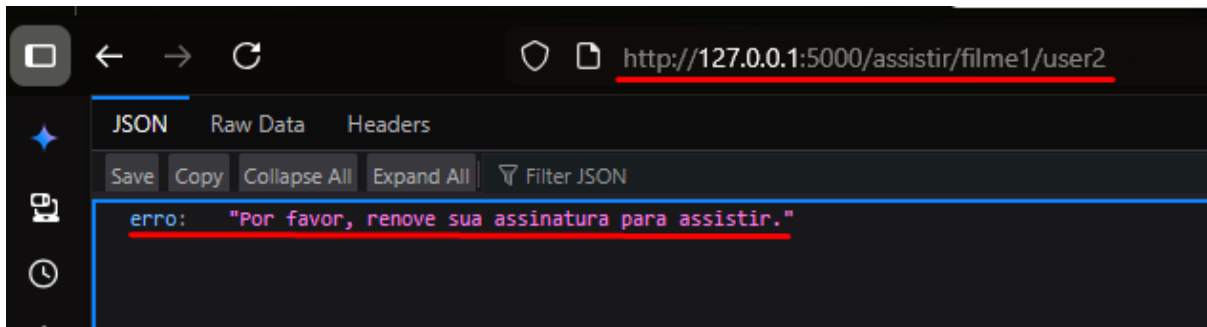
Acesse: <http://localhost:5000/assistir/filme1/user1>

iremos receber o json:

Acesse: <http://localhost:5000/assistir/filme1/user2>



<http://localhost:5000/assistir/filme1/user2>



Parte 4: Limpando a Casa

Quando a aula acabar, em vez de matar processo por processo, você pode usar um comando que derruba e deleta todos os containers de uma só vez, o comando é:

```
docker-compose down
```

O que o comando acima faz?

- Remove os containers.
- Remove a rede que o Compose criou automaticamente.
- Deixa o computador limpo.

Resumo para o Quadro (Tabela Comparativa)

A tabela a seguir contrasta o método manual que empregamos na aula anterior com o método Compose que acabamos de aprender. É notável como o Docker Compose simplifica significativamente nosso trabalho, especialmente ao lidar com um grande número de contêineres.

Ação	Modo Manual (Antigo)	Modo Compose (Novo)
Criar Rede	<code>docker network create ...</code>	<i>(Automático)</i>
Build	3x <code>docker build ...</code>	<code>docker-compose build</code>
Rodar	3x <code>docker run -d -p ... --net ...</code>	<code>docker-compose up</code>
Parar	3x <code>docker stop ...</code>	<code>docker-compose down</code>

Nomes DNS	Definidos manualmente na flag <code>--name</code>	Definidos no arquivo <code>.yaml</code>
------------------	---	---

Aqui está a tabela com os comandos essenciais do `docker-compose` para vocês utilizarem no dia a dia.

Comandos Básicos do Docker Compose

Comando	O que faz?	Quando usar?
<code>docker-compose up</code>	Sobe todos os serviços e mostra os logs no terminal.	Na primeira vez que for rodar ou para ver erros em tempo real. (Trava o terminal).
<code>docker-compose up -d</code>	Sobe tudo em segundo plano (Detached).	Quando você quer liberar o terminal e deixar o sistema rodando "escondido".
<code>docker-compose up --build</code>	Reconstrói as imagens antes de subir.	Essencial! Use sempre que você alterar o código Python (<code>.py</code>) ou o <code>Dockerfile</code> .
<code>docker-compose down</code>	Para e remove os containers e a rede criada.	Para desligar tudo de forma limpa ao fim da aula.
<code>docker-compose start</code>	Inicia os containers que já foram criados (mas estão parados).	Para retomar o trabalho sem recriar tudo do zero.
<code>docker-compose stop</code>	Para os containers sem removê-los.	Para dar uma pausa rápida sem perder o estado dos containers.

docker-compose logs -f	Mostra os logs de todos os serviços em tempo real.	Útil quando você rodou com -d e agora quer ver o que está acontecendo.
docker-compose ps	Lista os containers ativos e suas portas.	Para conferir se algum serviço morreu ou qual porta ele está usando.
docker-compose restart	Reinicia todos (ou um) serviço específico.	Se um serviço travou ou você quer forçar uma reinicialização rápida.



Dica de Ouro

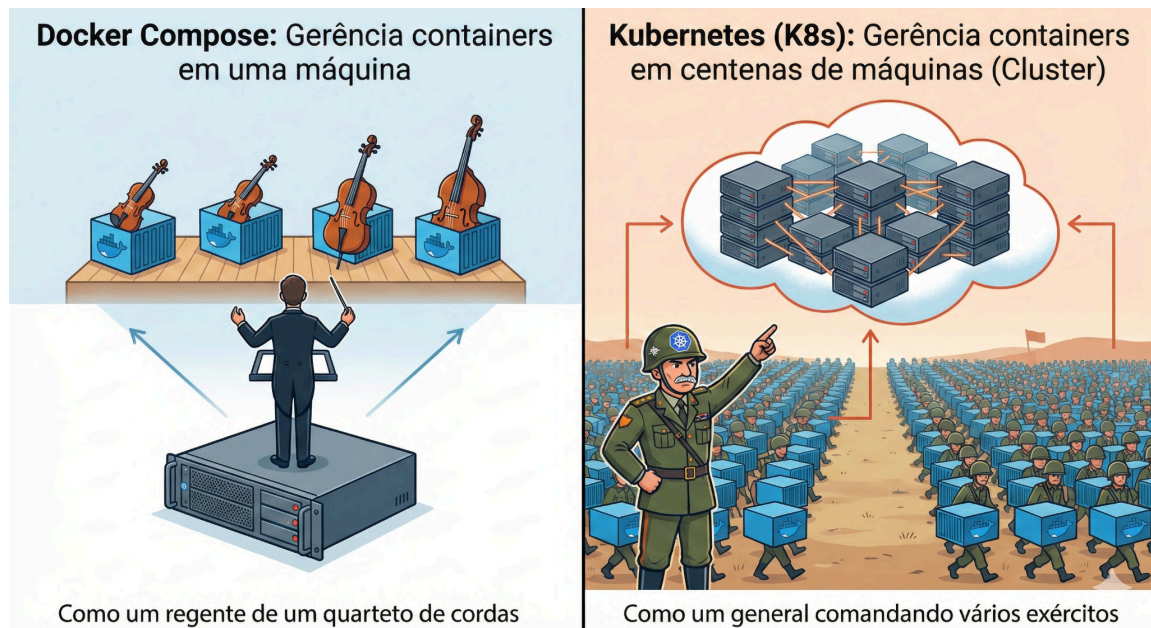
O erro mais comum é alterar o código e esquecer de atualizar o container. Crie o hábito de usar este combo:

Derruba o antigo, reconstrói com o código novo e sobe novamente

```
docker-compose down && docker-compose up --build
```

2. Kubernetes (K8s)

Agora que já entendemos que o **Docker** cria a "caixa" (Container) e o **Docker Compose** organiza essas caixas agrupando elas, é hora de apresentar o **Kubernetes (K8s)**.



A principais diferenças entre eles:

- **Docker Compose:** Gerência containers em **uma máquina**. (Como um regente de um quarteto de cordas).
- **Kubernetes:** Gerência containers em **centenas de máquinas (Cluster)**. (Como um general comandando vários exércitos).

1. O Conceito: O Capitão do Porto

Imagine que o Docker é o **Container** de carga e o Docker Compose é o **Guindaste** que coloca tudo em cima de um navio.

Mas e se o navio afundar? Ou se tiver carga demais para um navio só?

O **Kubernetes** é o **Capitão do Porto**. Ele não se importa com qual navio (servidor) vai levar a carga.

- **Auto-Healing (Cura):** Se um container "morre" (dá erro), o K8s percebe e cria outro novo imediatamente.
- **Scaling (Escala):** Se mil alunos entrarem na Netflix ao mesmo tempo, o K8s cria automaticamente 50 cópias do Gateway para aguentar o tranco.

2. Traduzindo Docker Compose para Kubernetes

Em Kubernetes, a configuração não se limita a um único arquivo. Normalmente, cada serviço é definido em um arquivo com a extensão **.yaml**, e cada um desses arquivos é estruturado em torno de dois conceitos fundamentais, ou seja, duas partes principais.

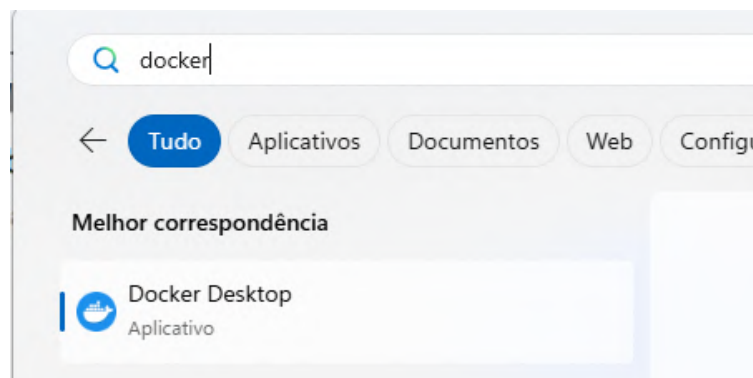
Essas duas partes são:

1. **Deployment (O Gerente):** Define o que rodar e quantas cópias.
 - "Quero 3 cópias do Gateway rodando sempre."
2. **Service (A Placa de Endereço):** Define onde encontrar.
 - "Quem procurar pelo nome 'gateway' vai ser atendido por uma das 3 cópias."

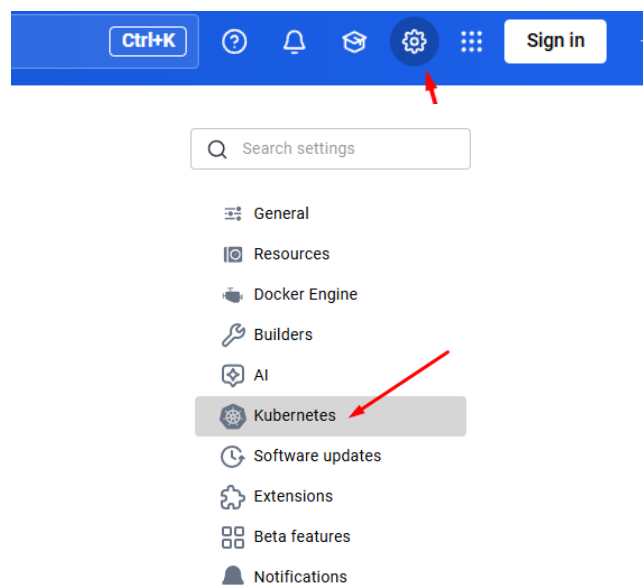
3. Instalando o kubernetes no docker-desktop:

O Docker Desktop vem com um Kubernetes embutido, mas ele vem **desligado** por padrão. Para ativá-lo siga esses passos:

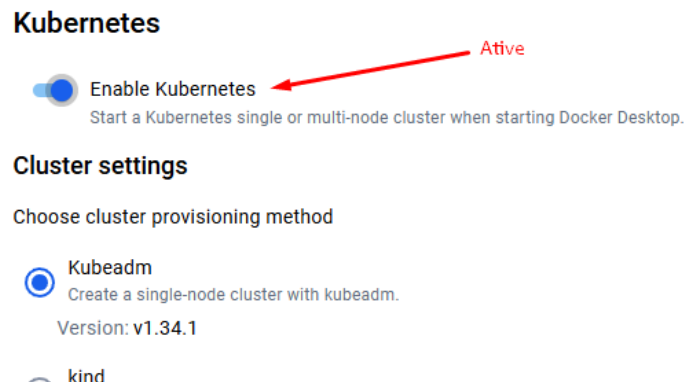
1. Abra o Docker Desktop.



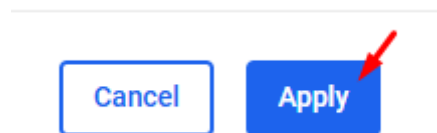
2. Vá em **Settings (Engrenagem) -> Kubernetes**.



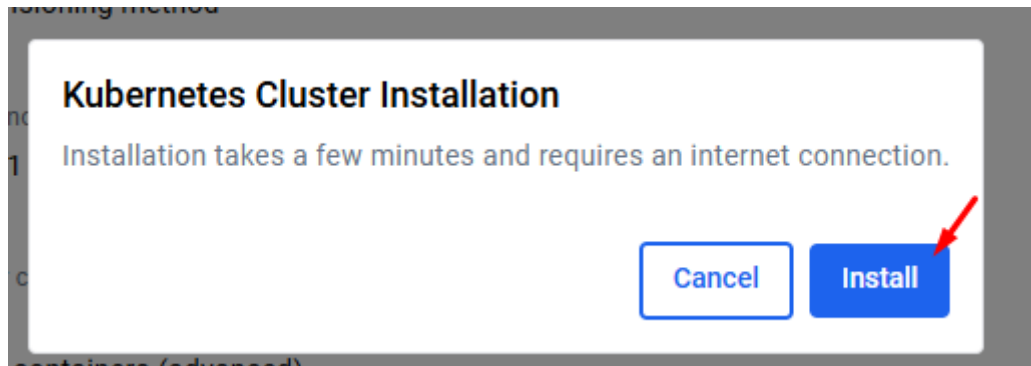
3. Marque a caixinha **Enable Kubernetes**.



4. Clique em **Apply**.



5. Após clicar em Apply aparecerá uma aviso para instalar o kubernetes

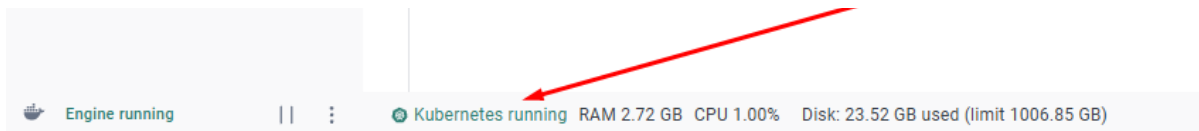


6. Espere a instalação terminar

Cluster



7. Quando a instalação terminar, uma luzinha do Kubernetes (canto inferior esquerdo) irá ficar verde.



Como verificar se funcionou?

É essencial verificar a instalação correta do Kubernetes (K8s) antes de avançar para a sua execução. Para esta checagem, utilize o comando:

```
kubectl cluster-info
```

A imagem a seguir exibe o resultado da execução do comando "**kubectl cluster-info**" no meu computador.

```
PowerShell 7.5.4
PS C:\Users\kelvi> kubectl cluster-info
Kubernetes control plane is running at https://kubernetes.docker.internal:6443
CoreDNS is running at https://kubernetes.docker.internal:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\kelvi> |
```

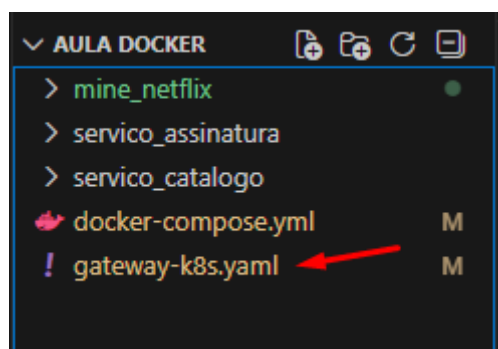
Kubernetes controle plane estão funcionando corretamente...

- **Se der erro:** O cluster ainda está desligado.
- **Se aparecer um texto colorido:** "Kubernetes control plane is running at...", então o sistema está pronto para receber o seu arquivo **gateway-k8s.yaml**.

4. Prática: Criando os arquivos .yaml

Vamos criar a configuração para o nosso **Gateway** (aquele da porta 5000).

Crie um arquivo chamado **gateway-k8s.yaml** no mesmo local do arquivo **docker-compose.yml**



e cole o código abaixo (o código foi anotado com comentários para detalhar as seções cruciais):

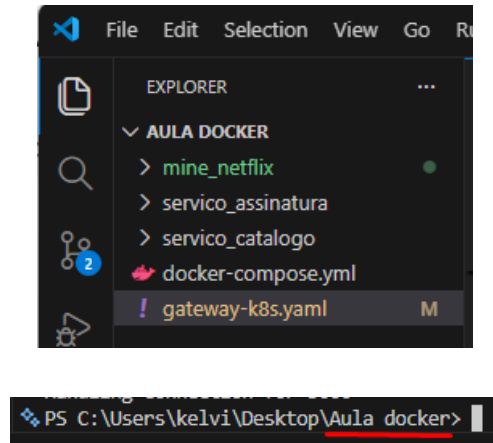
```
# 1. DEPLOYMENT: Quem roda o código
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gateway-deployment
spec:
  replicas: 3 # MÁGICA: Vamos rodar 3 cópias iguais ao mesmo tempo!
  selector:
    matchLabels:
      app: gateway-netflix
  template:
    metadata:
      labels:
        app: gateway-netflix
    spec:
      containers:
        - name: gateway-container
          # Aqui usamos a imagem que criamos com Docker
          image: minha-netflix-gateway:latest
          imagePullPolicy: Never # Diga ao Kubernetes para usar a imagem local
          (sem tentar baixar do Docker Hub)
          ports:
            - containerPort: 5000 # A porta que definimos no Python

---
# 2. SERVICE: Quem recebe o tráfego de rede
apiVersion: v1
kind: Service
metadata:
  name: gateway-service # Esse é o "DNS" que os outros usam
spec:
  type: LoadBalancer # Cria um IP externo para acessarmos do navegador
  selector:
    app: gateway-netflix
  ports:
    - protocol: TCP
      port: 5000 # Porta padrão da web (para não precisarmos digitar :5000)
      targetPort: 5000 # Redireciona para a porta 5000 do container
```

4. Como rodar o `gateway-k8s.yaml`?

Comandos:

1. Entre no terminal e vá para o mesmo diretório do arquivo `gateway-k8s.yaml`



2. Recrie e imagem do serviço `mine_netflix`:

```
docker build -t minha-netflix-gateway:latest ./mine_netflix
```

3. Rode o comando:

```
kubectl apply -f gateway-k8s.yaml
```

Esse comando irá aplicar a configuração em `gateway-k8s.yaml` (O K8s lê o arquivo e começa a trabalhar para subir as 3 cópias).

4. Ver os Pods (Containers): `kubectl get pods`

Resultado esperado:

```
PS C:\Users\kelvi\Desktop\Aula docker> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
gateway-deployment-75776c98bb-5dttq	1/1	Running	0	6m1s
gateway-deployment-75776c98bb-pthx6	1/1	Running	0	5m58s
gateway-deployment-75776c98bb-w7znd	1/1	Running	0	5m59s

A imagem ilustra três gateways em execução, indicando que o Kubernetes está gerenciando três contêineres de `gateway`. Essa configuração garante a alta disponibilidade: se um dos `gateways` falhar, os outros dois assumem automaticamente as operações.

Testar no Localhost

No Kubernetes, o "localhost" não funciona direto automaticamente. Você precisa criar um **Túnel** ou um **Redirecionamento**.

A forma mais fácil é usando o **port-forward** (redirecionamento de porta).

1. Rode este comando para ligar o seu localhost ao serviço do Kubernetes:

```
kubectl port-forward service/gateway-service 5000:5000
```

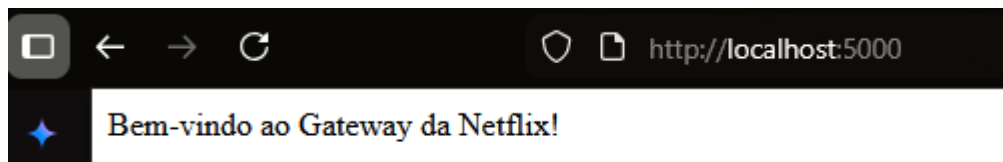
(Isso diz: "Tudo que chegar no meu localhost:5000, mande para a porta 5000 do serviço gateway").

Por exemplo, rodei o comando acima no terminal e agora o kubernetes está escutando na porta 5000:

```
PS C:\Users\kelvi\Desktop\Aula docker> kubectl port-forward service/gateway-service 5000:5000
Forwarding from 127.0.0.1:5000 -> 5000
```

2. **Teste no Navegador:** Agora você pode acessar: <http://localhost:5000>

Acessando <http://localhost:5000> aparecerá:



Note que não foi criado o arquivo **gateway-k8s.yaml** para os serviços de catálogo e assinatura. Consequentemente, ao tentar acessar a rota **<http://localhost:5000/filme1/user1>**, ocorrerá um erro, visto que esses serviços ainda não estão disponíveis.

Para criar as configurações do Kubernetes para esses serviços, basta seguir o mesmo procedimento já detalhado, adaptando os comandos conforme a necessidade de cada um.

Agora vamos simular um "acidente":

Se rodarmos do terminal o comando “**kubectl get pods**” teremos 3 serviços do gateway funcionando. Vamos deletar o serviço de nome “gateway-deployment-75776c98bb-5dttq”.

```
PS C:\Users\kelvi\Desktop\Aula docker> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
gateway-deployment-75776c98bb-5dttq 1/1     Running   1 (2m32s ago) 16h
gateway-deployment-75776c98bb-pthx6 1/1     Running   1 (2m32s ago) 16h
gateway-deployment-75776c98bb-w7znd 1/1     Running   1 (2m32s ago) 16h
PS C:\Users\kelvi\Desktop\Aula docker>
```

Para deletar esse serviço digite:

```
kubectl delete pod gateway-deployment-75776c98bb-5dttq
```

```
PS C:\Users\kelvi\Desktop\Aula docker> kubectl delete pod gateway-deployment-75776c98bb-5dttq
pod "gateway-deployment-75776c98bb-5dttq" deleted from default namespace
```

Se rodarmos novamente o comando “**kubectl get pods**” veremos o kubernetes substituindo esse serviço por outro de nome: **gateway-deployment-75776c98bb-fm5xb**

```
PS C:\Users\kelvi\Desktop\Aula docker> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
gateway-deployment-75776c98bb-fm5xb 1/1     Running   0           67s
gateway-deployment-75776c98bb-pthx6 1/1     Running   1 (13m ago) 16h
gateway-deployment-75776c98bb-w7znd 1/1     Running   1 (13m ago) 16h
PS C:\Users\kelvi\Desktop\Aula docker>
```

O K8s já criou um novo automaticamente para substituir o morto.

Chegamos ao fim da nossa aula. Espero que o conteúdo apresentado tenha sido de grande valia.